

Monitoring & Observability

Complete Interview Preparation Guide

Prometheus · Grafana · Datadog · CloudWatch · ELK · Kubernetes · AWS

Covers: Fundamentals · PromQL · Alerting · Incident Response · Troubleshooting Scenarios

Section	Topic
1	Monitoring Fundamentals
2	Core Monitoring Concepts (SLI, SLO, SLA, MTTR)
3	Metrics – Types, Cardinality, Key Infrastructure Metrics
4	Prometheus – Architecture, Exporters, Service Discovery
5	PromQL – Queries, rate(), irate(), Practical Examples
6	Grafana – Dashboards, Panels, Alerts
7	Kubernetes Monitoring – Nodes, Pods, EKS
8	AWS CloudWatch – Metrics, Logs, Alarms, Insights

9	Logging – ELK Stack, Structured Logs
10	Datadog – Agents, Alerts, Integrations
11	Alerting Best Practices
12	Application Monitoring
13	Incident Response & Troubleshooting Scenarios

Section 1: Monitoring Fundamentals

Q1. What is Monitoring?

Monitoring is the process of continuously collecting, tracking, and analysing data about a system to understand its health, performance, and availability.

- It tells you WHAT is happening in your system right now.
- Example: Checking if CPU > 80%, memory usage, HTTP error rates.

✓ **Monitoring = watching known metrics for expected problems.**

Q2. Why is Monitoring Important?

- Detects problems before users are impacted.
- Helps measure system reliability and performance.
- Enables data-driven decisions for capacity planning.
- Reduces Mean Time To Detect (MTTD) and Mean Time To Recover (MTTR).
- Provides historical data for trend analysis and audits.

Q3. What is Observability?

Observability is the ability to understand the internal state of a system by examining its external outputs (metrics, logs, traces).

- It answers WHY something is happening, not just what.
- A system is observable if you can diagnose any failure without deploying new code.

✓ **Observability = deep insight into unknown or unexpected failures.**

Q4. Difference Between Monitoring and Observability?

Monitoring

- Reactive — checks known failure conditions.
- Works well for predictable systems.
- Example: Alert if CPU > 90%.

Observability

- Proactive — helps explore unknown failures.
- Needed for complex, distributed, microservices systems.
- Example: Why is one microservice slow only for certain users?

✓ **Monitoring is a subset of Observability.**

Q5. What are the Three Pillars of Observability?

- **Metrics** – Numerical measurements over time (CPU %, request rate).
- **Logs** – Timestamped text records of events that happened.
- **Traces** – Track a single request across multiple services (distributed tracing).

Q6. What are Metrics?

Metrics are numerical data points collected at regular intervals. They represent the state or behaviour of a system over time.

- Example: CPU utilisation = 72%, Memory used = 4 GB, Requests/sec = 1200.

Q7. What are Logs?

Logs are time-stamped text records that describe what happened inside an application or system.

- Example: '2024-06-01 10:00:01 ERROR – DB connection timeout'.
- Used for debugging, auditing, and root cause analysis.

Q8. What are Traces?

Traces track the journey of a single request as it travels through multiple services in a distributed system.

- Each step is called a **Span**; all spans together form a Trace.
- Tools: Jaeger, Zipkin, AWS X-Ray, Datadog APM.

✓ **Traces answer: Which service is causing high latency for this request?**

Q9. What is Telemetry Data?

Telemetry data is any automatically collected data about the behaviour and performance of a system.

- Includes metrics, logs, traces, and events.
- Sent automatically from systems to a central monitoring platform.

Q10. Why Do Modern Systems Need Observability?

- Modern systems are distributed, cloud-native, and use microservices.
- Failures can happen across many services simultaneously.
- Traditional monitoring cannot handle unknown failure modes.
- Observability allows teams to ask new questions about failures without deploying new instrumentation.

Section 2: Core Monitoring Concepts

Q1. What is a KPI (Key Performance Indicator)?

A KPI is a business-level metric that measures how well a team or service is achieving its objectives.

- Example: 99.9% uptime, < 200 ms response time, < 0.1% error rate.

Q2. What is an SLI (Service Level Indicator)?

An SLI is a specific, measurable metric that indicates the level of service being provided.

- Example: The percentage of HTTP requests that return a 200 OK response.
- SLIs are the actual measurements you track.

Q3. What is an SLO (Service Level Objective)?

An SLO is an internal target for an SLI. It defines how good the service should be.

- Example: 99.9% of requests should return 200 OK within 300 ms.
- SLOs are set by the engineering team and guide alert thresholds.

Q4. What is an SLA (Service Level Agreement)?

An SLA is a contractual agreement between a service provider and a customer defining the expected level of service.

- Example: We guarantee 99.5% uptime per month. If we fail, the customer gets a credit.
- SLAs have financial/legal penalties if violated.

Q5. Difference Between SLI, SLO, and SLA?

Simple Analogy

- **SLI** = Your actual speedometer reading (e.g., 65 mph).
- **SLO** = Internal target speed (e.g., stay under 70 mph).
- **SLA** = Legal commitment to passengers (e.g., arrive in 2 hours or refund).

✓ **SLI measures it. SLO targets it. SLA promises it.**

Q6. What is MTTR (Mean Time to Recovery)?

MTTR is the average time taken to restore a service after a failure.

- Formula: Total downtime / Number of incidents.
- Lower MTTR = faster recovery = better reliability.

✓ **Monitoring helps reduce MTTR by quickly pinpointing root cause.**

Q7. What is MTTD (Mean Time to Detect)?

MTTD is the average time between when a problem starts and when the team detects it.

- Good monitoring and alerting reduces MTTD.
- Example: If an error starts at 10:00 and an alert fires at 10:03, MTTD = 3 minutes.

Q8. What is Alert Fatigue?

Alert fatigue occurs when too many alerts fire — many of which are false positives — causing teams to ignore or delay responding to alerts.

- Results in missed real incidents.
- Caused by poorly tuned thresholds or monitoring everything without prioritisation.

✓ **Fix by reducing noise: use better thresholds, severity levels, and silence noisy alerts.**

Q9. What is a Monitoring Dashboard?

A monitoring dashboard is a visual interface that displays key metrics, logs, and alerts in one place for real-time system visibility.

- Examples: Grafana, Kibana, CloudWatch Dashboards, Datadog Dashboards.
- Should show: system health, resource utilisation, error rates, and latency.

Q10. What Metrics Would You Monitor for a Production Application?

Infrastructure

- CPU utilisation, Memory usage, Disk I/O, Network throughput.

Application

- Request rate (RPS), Error rate (%), Latency (p50, p95, p99).

Database

- Query latency, Active connections, Slow queries.

Business

- Active users, Orders per minute, Payment success rate.

Section 3: Metrics – Types, Cardinality & Key Infrastructure Metrics

Q1. What Metric Types Exist?

- **Counter** – Always increases. Tracks cumulative totals. (e.g., total HTTP requests).
- **Gauge** – Can go up or down. Tracks current values. (e.g., CPU %, active connections).
- **Histogram** – Tracks distribution of values in configurable buckets. (e.g., request duration).
- **Summary** – Similar to histogram but calculates quantiles on the client side.

Q2. Difference Between Counter and Gauge?

Counter

- Only increases (or resets to 0 on restart).
- Example: Total requests served = 100,000.

Gauge

- Can increase or decrease.
- Example: Current memory usage = 3.2 GB.

Q3. What is a Histogram?

A histogram samples observations and counts them in configurable buckets (ranges).

- Example: How many requests took 0–100 ms, 100–200 ms, 200–500 ms?
- Useful for calculating percentiles like p95, p99.

✓ Use histogram when you want to know the distribution of latency.

Q4. What is a Summary?

A Summary also samples observations like histograms, but calculates quantiles (percentiles) directly on the client/agent side.

- Quantiles are pre-calculated and cannot be aggregated across instances.
- Histogram is preferred in Prometheus because it allows server-side aggregation.

Q5. What is Time-Series Data?

Time-series data is a sequence of data points collected at regular time intervals.

- Each data point has a timestamp, a metric name, labels, and a value.
- Example: `cpu_usage{host='web-01'} 72.5 at 10:00:00.`

Q6. Why are Labels Important in Metrics?

Labels add dimensions to metrics, allowing you to filter and aggregate data by specific attributes.

- Example: `http_requests_total{status='200', method='GET', service='api'}`.
- Labels allow one metric to answer many different questions.

Q7. What Causes High Cardinality and Why is it a Problem?

High cardinality occurs when a metric has labels with many unique values — creating thousands or millions of unique time series.

- Example: Using `user_id` or IP address as a label = millions of series.
- Problem: Consumes massive memory and storage; slows down Prometheus.

✓ **Avoid high-cardinality labels like user IDs, session IDs, IP addresses.**

Q8. Key Metrics for EC2 (AWS)

- `CPUUtilization` – % CPU used.
- `NetworkIn` / `NetworkOut` – Network traffic.
- `DiskReadOps` / `DiskWriteOps` – Disk I/O activity.
- `StatusCheckFailed` – Instance health check.

Q9. Key Metrics for Kubernetes

- Pod CPU / Memory requests and limits.
- `container_restart_count` – Number of restarts.
- `kube_pod_status_phase` – Running / Pending / Failed.
- Node CPU utilisation, Node memory pressure.

Q10. Key Metrics for Database

- Query latency (avg, p95, p99).
- Active / idle connections.
- Slow query count.
- Replication lag (for replicas).
- Cache hit ratio.

Q11. Key Metrics for Application / Load Balancer

Application

- Request rate (RPS), Error rate (4xx/5xx %), Response latency.

Load Balancer

- Healthy / Unhealthy target count, Request count, 5xx errors, Latency.

Section 4: Prometheus – Architecture, Exporters & Service Discovery

Q1. What is Prometheus?

Prometheus is an open-source monitoring and alerting system. It collects and stores time-series metrics by scraping HTTP endpoints.

- Built by SoundCloud, now a CNCF graduated project.
- Works best with Kubernetes and cloud-native environments.

Q2. How Does Prometheus Work? (Architecture)

- **Scraping:** Prometheus pulls metrics from targets (apps, exporters) by making HTTP GET requests to /metrics endpoints.
- **Storage:** Stores metrics in its local time-series database (TSDB).
- **PromQL:** Query language to retrieve and analyse metrics.
- **Alertmanager:** Handles routing, grouping, and silencing of alerts.
- **Service Discovery:** Automatically discovers targets in Kubernetes, EC2, etc.

✓ **Prometheus is PULL-based — it goes to fetch data, unlike push-based systems.**

Q3. What is Scraping?

Scraping is the process where Prometheus makes an HTTP GET request to a target's /metrics endpoint at a configured interval.

- Default scrape interval: 15–30 seconds.
- Each scrape collects all metrics exposed by that target.

Q4. What are Exporters?

Exporters are agents that collect metrics from third-party systems (that don't natively expose Prometheus metrics) and convert them to Prometheus format.

- **Node Exporter** – Hardware and OS metrics (CPU, Memory, Disk).
- **Blackbox Exporter** – Probes HTTP, DNS, TCP endpoints.
- **MySQL Exporter** – Database metrics.
- **cAdvisor** – Container metrics for Kubernetes.

Q5. What is a Target?

A target is any service or application endpoint that Prometheus scrapes for metrics.

- Defined in `prometheus.yml` under `scrape_configs`.
- Can be static (hard-coded IPs) or dynamic (via service discovery).

Q6. What is Alertmanager?

Alertmanager is a component that handles alerts sent by Prometheus. It performs deduplication, grouping, routing, and notification.

- Routes alerts to Slack, PagerDuty, email, etc.
- Supports silencing (mute during maintenance) and inhibition (suppress low-priority alerts when a major one fires).

Q7. How Does Service Discovery Work in Prometheus?

Service discovery allows Prometheus to automatically find and monitor new targets without manual configuration.

- **Kubernetes SD:** Discovers Pods, Services, Nodes dynamically.
- **EC2 SD:** Discovers EC2 instances automatically.
- When a new Pod starts, Prometheus automatically scrapes it.

Q8. What are Recording Rules?

Recording rules pre-compute frequently used or expensive PromQL expressions and store the results as new metrics.

- Improves query performance — especially for dashboards.
- Example: Pre-compute job-level CPU average across all Pods.

Q9. What is Node Exporter?

Node Exporter is a Prometheus exporter that collects hardware and OS-level metrics from Linux/Unix hosts.

- Exposes: CPU usage, memory, disk I/O, filesystem, network stats.
- Runs as a DaemonSet in Kubernetes to collect metrics from every node.

Q10. What is kube-state-metrics?

kube-state-metrics is a Kubernetes add-on that exposes the state of Kubernetes objects (not resource usage) as Prometheus metrics.

- Tracks: Deployment replicas, Pod phases, Node conditions, Job statuses.
- Complement to Node Exporter (which tracks resource usage).

Q11. What are ServiceMonitor and PodMonitor?

ServiceMonitor

- A Kubernetes CRD (Custom Resource Definition) used by Prometheus Operator to define which Services to scrape.

PodMonitor

- Similar to ServiceMonitor but targets Pods directly, without needing a Kubernetes Service.

Q12. What are Prometheus Limitations?

- Not designed for long-term storage (default retention: 15 days).
- Single-node architecture; doesn't scale horizontally natively.
- High-cardinality metrics cause memory and performance issues.
- Not ideal for log storage or distributed tracing.

✓ **Solutions: Thanos or Cortex for long-term storage and horizontal scaling.**

Q13. What is Remote Write?

Remote Write is a Prometheus feature that continuously sends (pushes) metrics to an external storage system.

- Used with Thanos, Cortex, or managed services like Grafana Cloud.
- Solves the single-node storage limitation.

Section 5: PromQL – Queries, Functions & Practical Examples

Q1. What is PromQL?

PromQL (Prometheus Query Language) is the query language used to select and aggregate time-series data stored in Prometheus.

- Used in Grafana panels and Prometheus alert rules.

Q2. How Do You Query CPU Usage?

```
rate(container_cpu_usage_seconds_total[5m])
```

- Calculates the per-second CPU usage rate over the last 5 minutes.

```
100 - (avg by(instance)(rate(node_cpu_seconds_total{mode='idle'}[5m])) * 100)
```

- CPU utilisation % per node (100% minus idle time).

Q3. How Do You Calculate Memory Utilisation?

```
(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) /  
node_memory_MemTotal_bytes * 100
```

- Returns memory usage percentage on a node.

Q4. What Does rate() Do?

rate() calculates the per-second average rate of increase of a counter over a specified time window.

- Designed for counters — smoothed average over the window.
- Example: rate(http_requests_total[5m]) = average requests per second over 5 min.

Q5. What Does irate() Do?

irate() calculates the per-second instantaneous rate using only the last two data points.

- More sensitive to spikes — reacts faster than rate().

✓ Use rate() for dashboards (smoother). Use irate() for alerting on sudden spikes.

Q6. Difference Between rate() and irate()?

- **rate()** – Smoothed average over the full range. Good for trend visibility.
- **irate()** – Instantaneous rate using last 2 points. Good for spike detection.

Q7. What Does sum() Do?

sum() aggregates (adds up) values across multiple time series.

```
sum(rate(http_requests_total[5m])) by (service)
```

- Sums request rate per service.

Q8. What Does increase() Do?

increase() returns the total increase in a counter over the specified time range.

```
increase(http_requests_total[1h])
```

- Total number of HTTP requests in the last 1 hour.

Q9. Explain: rate(container_cpu_usage_seconds_total[5m])

- **container_cpu_usage_seconds_total** – Cumulative counter of CPU seconds used by containers.
- **rate(...[5m])** – Calculates per-second CPU usage averaged over 5 minutes.
- **Result:** CPU cores used per second per container.

✓ **Multiply by 100 to get CPU % if comparing to limits.**

Q10. How Do You Identify High CPU Pods?

```
topk(5, sum(rate(container_cpu_usage_seconds_total[5m])) by (pod))
```

- Returns the top 5 pods with highest CPU usage.

Section 6: Grafana – Dashboards, Panels, Alerts & Variables

Q1. What is Grafana?

Grafana is an open-source data visualisation and analytics platform. It connects to multiple data sources (Prometheus, CloudWatch, ELK, Loki, etc.) and displays data as dashboards.

✓ **Grafana visualises; Prometheus stores.**

Q2. What are Dashboards, Panels, and Variables?

Dashboards

- A collection of panels organised on a single page to provide a unified view.

Panels

- Individual visualisation units inside a dashboard (graph, stat, table, gauge).

Variables

- Dynamic dropdown filters that let users switch between environments, namespaces, or services without editing queries.

Q3. How Do Grafana Alerts Work?

- Define alert rules on a panel query with a threshold condition.
- Grafana evaluates the query at a configured interval.
- If the condition is met (e.g., CPU > 90%), an alert fires.
- Notifications are sent via contact points: Slack, PagerDuty, email.

Q4. Kubernetes Dashboard – What Would You Display?

Cluster Overview

- Total nodes, Running / Pending / Failed pods, Resource utilisation %.

Node Metrics

- CPU usage per node, Memory usage per node, Disk I/O.

Pod Metrics

- Restart count, CPU/Memory per pod, Pod status.

Alerts Panel

- Active firing alerts table.

Q5. How Do You Troubleshoot Dashboard Issues?

- Check if the data source is connected and reachable.
- Verify the PromQL query returns data using Explore mode.
- Check time range — wrong range = no data.
- Check variable values are correctly set.
- Inspect query errors in the panel inspector.

Section 7: Kubernetes Monitoring – Nodes, Pods, Deployments & EKS

Q1. How Do You Monitor Kubernetes?

- **Prometheus + Grafana Stack:** Most common. Deploy via kube-prometheus-stack Helm chart.
- **Node Exporter:** Collects node-level hardware metrics.
- **kube-state-metrics:** Exposes Kubernetes object states.
- **cAdvisor:** Built into kubelet, provides container resource metrics.
- **CloudWatch Container Insights:** For EKS managed clusters.

Q2. What Kubernetes Metrics are Most Important?

Node Level

- CPU utilisation, Memory utilisation, Disk pressure, Network I/O.

Pod Level

- CPU requests vs limits, Memory requests vs limits, Restart count.

Cluster Level

- Total pod count, Running vs Pending vs Failed pods, PVC utilisation.

Q3. What Metrics Indicate a Failing Pod?

- **High restart count** (CrashLoopBackOff) – kube_pod_container_status_restarts_total.
- **OOMKilled** – Pod killed due to exceeding memory limit.
- **Pending status** – Unable to schedule (insufficient resources).
- **High CPU throttling** – CPU usage consistently hitting limits.

Q4. How Do You Monitor a Kubernetes Node That Becomes NotReady?

- Alert on: kube_node_status_condition{condition='Ready', status='true'} == 0.
- Check node events: kubectl describe node .
- Check kubelet logs on the node.
- Check for disk pressure, memory pressure, or network issues.

✓ **Grafana alert + PagerDuty ensures on-call is notified immediately.**

Q5. How Do You Monitor EKS (AWS)?

- Enable **CloudWatch Container Insights** for cluster-level and pod-level metrics.
- Deploy **Prometheus + Grafana** stack inside the EKS cluster.
- Use **AWS Managed Prometheus** + Grafana for fully managed setup.
- Monitor EKS control plane logs via CloudWatch Logs.
- Use **Datadog Agent** with Kubernetes integration for combined infra + app monitoring.

Section 8: AWS CloudWatch – Metrics, Logs, Alarms & Insights

Q1. What is CloudWatch?

Amazon CloudWatch is AWS's native monitoring service. It collects metrics, logs, and events from AWS services and applications.

- Supports metrics, logs, alarms, dashboards, and anomaly detection.

Q2. What are CloudWatch Metrics, Logs, and Alarms?

Metrics

- Numerical time-series data from AWS services (EC2 CPU, RDS connections, etc.).

Logs

- Application and infrastructure logs stored centrally in Log Groups.

Alarms

- Automated alerts triggered when a metric crosses a threshold. Actions include SNS notification, Auto Scaling, Lambda.

Q3. What Metrics Do You Monitor for EC2?

- CPUUtilization – % of CPU capacity used.
- NetworkIn / NetworkOut – Network bytes in/out.
- DiskReadOps / DiskWriteOps – Number of disk operations.
- StatusCheckFailed – Indicates hardware or OS issue.

✓ **Install CloudWatch Agent for memory and disk utilisation (not collected by default).**

Q4. What Metrics Do You Monitor for EKS via CloudWatch?

- pod_cpu_utilization, pod_memory_utilization (Container Insights).
- cluster_node_count, cluster_failed_node_count.
- service_number_of_running_pods.

Q5. What is CloudWatch Insights (Logs Insights)?

CloudWatch Logs Insights is an interactive log querying service that lets you analyse logs in CloudWatch using a SQL-like query language.

```
fields @timestamp, @message | filter @message like /ERROR/ | sort @timestamp desc | limit 50
```

- Above query finds the 50 most recent ERROR log lines.

Q6. What are Custom Metrics in CloudWatch?

Custom metrics allow applications and scripts to send their own metrics to CloudWatch beyond what AWS services provide automatically.

- Example: Application error count, queue depth, active user sessions.
- Sent via CloudWatch Agent, AWS SDK, or the PutMetricData API.

Q7. What is Anomaly Detection in CloudWatch?

CloudWatch Anomaly Detection uses ML to automatically establish a normal baseline for a metric and alerts when the metric deviates from that baseline.

- Useful for metrics with variable normal ranges (e.g., traffic higher on weekdays).
- No manual threshold configuration needed.

Section 9: Logging, ELK Stack & Structured Logging

Q1. What is the ELK Stack?

- **E – Elasticsearch:** Distributed search and analytics engine that stores and indexes logs.
- **L – Logstash:** Data processing pipeline that ingests, transforms, and ships logs.
- **K – Kibana:** Web interface for visualising and querying logs in Elasticsearch.
- **Filebeat:** Lightweight agent that ships log files from servers to Logstash or Elasticsearch directly.

Q2. How Does the ELK Stack Work?

- Applications write logs to files or stdout.
- **Filebeat** reads log files and ships them to Logstash.
- **Logstash** parses, filters, and enriches log data.
- **Elasticsearch** indexes and stores the logs.
- **Kibana** provides the dashboard to search and visualise logs.

Q3. What are Shards and Replicas in Elasticsearch?

Shards

- An index is split into multiple shards. Each shard is a self-contained Lucene index.
- More shards = better parallelism for large data.

Replicas

- Copies of shards on different nodes for fault tolerance and read performance.

✓ **Rule of thumb: Shard size 10–50 GB each. Start with 1 replica per shard.**

Q4. What is Index Lifecycle Management (ILM)?

ILM automates the management of index lifecycle in Elasticsearch: Hot → Warm → Cold → Delete.

- **Hot:** Recent data, active indexing and querying.
- **Warm:** Older data, less frequent queries.
- **Cold:** Rarely accessed, stored cheaply.
- **Delete:** Automatically deleted after retention period.

Q5. Why Use Structured / JSON Logging?

Structured logging uses a consistent format (usually JSON) for log messages, making them machine-parseable.

```
{"level":"ERROR","timestamp":"2024-06-01T10:00:01Z","service":"api","message":"DB timeout","duration_ms":3000}
```

- Easier to search, filter, and aggregate in ELK / Datadog.
- No regex parsing required.

✓ **Always log with context: service name, request ID, user ID, error type.**

Q6. What Log Levels Exist?

- **DEBUG** – Detailed info for development/debugging. Never in production.
- **INFO** – Normal operational messages.
- **WARN** – Unexpected situations that are not errors yet.
- **ERROR** – Errors that need attention but don't stop the application.
- **FATAL / CRITICAL** – Severe errors that crash the application.

Q7. What Causes Slow Elasticsearch Performance?

- Too many shards (over-sharding) wasting resources.
- Unoptimised queries doing full-index scans.
- JVM heap pressure or GC pauses.
- Disk I/O bottleneck on high ingest rates.
- Missing index mapping (dynamic mapping creates excessive fields).

Section 10: Datadog – Agents, Dashboards, Alerts & Integrations

Q1. What is Datadog?

Datadog is a cloud-based monitoring and observability platform. It collects metrics, logs, and traces from infrastructure, applications, and cloud services in one unified platform.

✓ **Datadog = Metrics + Logs + Traces + APM + Alerts — all in one SaaS platform.**

Q2. How Does the Datadog Agent Work?

The Datadog Agent is a lightweight software installed on hosts (servers, containers, Kubernetes nodes) that collects and ships data to Datadog's cloud platform.

- Collects: system metrics, logs, traces, and process data.
- For Kubernetes: deployed as a DaemonSet on all nodes.
- Uses integrations (plugins) to monitor databases, web servers, cloud services.

Q3. How Do You Integrate Kubernetes with Datadog?

- Deploy Datadog Agent as a DaemonSet using the official Helm chart.
- Enable Kubernetes State Metrics integration.
- Enable Log collection and APM tracing.
- Add annotations to Pods for auto-discovery:

```
ad.datadoghq.com/container-name.check_names: ["nginx"]
```

Q4. How Does Datadog Collect Logs?

- Datadog Agent tails log files or reads from container stdout/stderr.
- Logs can be forwarded to Datadog via Logstash, Fluentd, or AWS Lambda.
- Supports log parsing pipelines (Grok patterns) to structure raw logs.

Q5. How Do Datadog Alerts Work?

- **Metric Monitor** – Alert when a metric exceeds a threshold.
- **Log Monitor** – Alert on log patterns or anomalies.
- **Anomaly Monitor** – Alert on unusual metric behaviour using ML.
- **Composite Monitor** – Alert when multiple monitors fire simultaneously.
- Notifications sent to: Slack, PagerDuty, email, webhooks.

Section 11: Alerting Best Practices

Q1. What Makes a Good Alert?

- **Actionable** – Only alert if someone needs to do something right now.
 - **Accurate** – Low false positive rate.
 - **Context-rich** – Include what, why, where, and a runbook link.
 - **Appropriate severity** – Not everything is P1/Critical.
- ✓ **Rule: If an alert fires and no action is taken, either fix the system or delete the alert.**

Q2. How Do You Reduce False Positives and Alert Fatigue?

- Set thresholds based on historical baselines, not guesses.
- Use for clause in Prometheus: alert only if the condition persists for 5 min.
- Group related alerts using Alertmanager group_by.
- Silence alerts during maintenance windows.
- Regularly audit and prune unused or noisy alerts.

Q3. What Alerts Should Every Kubernetes Cluster Have?

- Node NotReady – `kube_node_status_condition{condition='Ready'} == 0`.
- Pod CrashLoopBackOff – `kube_pod_container_status_waiting_reason == 'CrashLoopBackOff'`.
- High Pod Restart Count – `container_restart_count > 5` in 1 hour.
- High CPU / Memory usage > 85% on nodes.
- PVC at > 80% capacity.
- Deployment replicas unavailable.

Q4. What Alerts Should Every AWS Environment Have?

- EC2 CPUUtilization > 80% for 15 min.
- RDS FreeStorageSpace < 10 GB.
- ALB 5xx error rate > 1%.
- Lambda error rate spike.
- Billing alarm when estimated charges exceed budget.
- CloudTrail: unauthorized API calls.

Q5. What is Alert Suppression / Silencing / Inhibition?

Silencing

- Temporarily mutes alerts during maintenance windows.

Inhibition

- Suppresses low-priority alerts when a higher-priority alert is already firing.
- Example: Suppress Pod alerts if the entire Node is already down.

Section 12: Application Monitoring – Latency, Throughput, Error Rate

Q1. What is Latency?

Latency is the time taken for a request to be processed and a response to be returned.

- **p50** – Median latency (50% of requests are faster than this).
- **p95** – 95% of requests are faster than this. Highlights most user experiences.
- **p99** – 99th percentile. Identifies the slowest requests (tail latency).

✓ Always monitor p95 and p99, not just averages — averages hide outliers.

Q2. What is Throughput?

Throughput is the number of requests a system can handle per unit of time.

- Measured in Requests Per Second (RPS) or Transactions Per Second (TPS).
- A sudden drop in throughput with no traffic decrease = application problem.

Q3. What is Error Rate?

Error rate is the percentage of requests that result in an error (4xx/5xx HTTP status codes).

```
rate(http_requests_total{status=~'5..'}[5m]) / rate(http_requests_total[5m]) * 100
```

- Alert if 5xx error rate > 1% sustained for 5 minutes.

Q4. What is Apdex?

Apdex (Application Performance Index) is a standard metric that measures user satisfaction with response time.

- **Satisfied:** Response time $\leq T$ (target time, e.g. 300 ms).
- **Tolerating:** Response time between T and $4T$.
- **Frustrated:** Response time $> 4T$ or error.
- Score: $(\text{Satisfied} + 0.5 * \text{Tolerating}) / \text{Total}$. Score of 1.0 = perfect.

Q5. How Do You Monitor Microservices?

- Monitor each service independently for: latency, error rate, throughput.
- Use distributed tracing (Jaeger / Zipkin / Datadog APM) to track cross-service requests.
- Monitor service-to-service communication (service mesh: Istio metrics).
- Set SLOs per service and alert on SLO burn rate.

Section 13: Incident Response & Troubleshooting Scenarios

Scenario 1: Users report the application is slow – What metrics would you check first?

- Check **Application latency** (p95, p99) in Grafana / Datadog dashboard.
- Check **Error rate** – are 5xx errors increasing?
- Check **CPU and Memory** on all application pods and nodes.
- Check **Database query latency** – is the DB the bottleneck?
- Check **Network latency and packet loss**.
- Look at **Application logs** for timeout errors or exceptions.
- Check **Load Balancer** request count and 5xx rate.

✓ **Start from the user-facing metrics and drill down: LB → App → DB → Infra.**

Scenario 2: CPU usage spikes to 95% – How would you investigate?

- Identify which **Node or Pod** is consuming high CPU.

```
topk(5, sum(rate(container_cpu_usage_seconds_total[5m])) by (pod))
```
- Check if it's a **sudden spike** (single bad request?) or sustained (code issue).
- Look at application logs for CPU-intensive operations.
- Check if a **cron job or batch process** started running.
- Use **kubectl top pods** for live CPU usage.
- Check if CPU limits are too low – causing throttling, not full CPU.

✓ **CPU throttling ≠ CPU spike. Check `cpu_throttled_seconds` metric.**

Scenario 3: Pods are restarting frequently – How would monitoring help identify the issue?

- Alert fires: **container_restart_count > 5** in 1 hour.
- Check **exit code** – Exit code 137 = OOMKilled, Exit code 1 = App crash.

```
kube_pod_container_status_last_terminated_reason
```
- If **OOMKilled**: increase memory limits or fix memory leak.
- If **CrashLoopBackOff**: check app logs for startup errors.
- Run: **kubectl describe pod <pod-name>** for events.
- Run: **kubectl logs <pod-name> --previous** for last crash logs.

Scenario 4: Database latency suddenly increases – What would you review?

- Check DB-specific metrics: **active connections**, **slow queries**, **lock waits**.
 - Check if **connection pool is exhausted** on the application side.
 - Check **CPU and Memory on the DB instance**.
 - Look at **slow query logs** in RDS/CloudWatch for long-running queries.
 - Check **Replication lag** if reads are going to a replica.
 - Look for **deployment events** – did a code change trigger a full table scan?
- ✓ **Most DB latency spikes are caused by missing indexes or connection exhaustion.**

Scenario 5: A Kubernetes Node Becomes NotReady – How Would You Detect and Troubleshoot?

- Alert fires: `kube_node_status_condition{condition='Ready', status='true'} == 0`.
- Run: **kubectl get nodes** to confirm node status.
- Run: **kubectl describe node <node-name>** – check Events section.
- Check for: **DiskPressure**, **MemoryPressure**, **PIDPressure**, **NetworkUnavailable**.
- SSH into the node and check **kubelet service status**: `systemctl status kubelet`.
- Check **Node Exporter metrics** for disk full, OOM conditions.
- For EKS: check EC2 instance status in AWS Console.

Scenario 6: Application returns HTTP 500 errors – How would you investigate?

- Check **Error rate metric** in Grafana – when did 5xx start?
- Check **Application logs** in ELK or CloudWatch Logs Insights for stack traces.

```
fields @timestamp, @message | filter @message like /ERROR|Exception/ | sort @timestamp desc
```
- Check if a **deployment happened recently** (`kubectl rollout history`).
- Check **DB connectivity** – connection refused errors in logs?
- Check **downstream service health** – is a dependency down?
- If recent deployment: **kubectl rollout undo deployment/<name>** to revert.

Scenario 7: Memory usage steadily increases – How would you identify a memory leak?

- Chart **container_memory_usage_bytes** over 24–48 hours in Grafana.
- Look for a **sawtooth pattern** (gradual increase, then GC drop) vs. a continuous upward trend.
- Check if memory usage **never decreases** even after traffic drops = strong signal of a leak.
- Enable **heap dump / profiling** on the application for specific analysis.
- Check for **caching without eviction** or unclosed connections in code.

✓ **Memory leaks are confirmed by: usage grows proportional to uptime, not traffic.**

Scenario 8: A CloudWatch Alarm Triggers at 3 AM – What Steps Would You Take?

- **Acknowledge** the alert in PagerDuty / Slack. Start incident channel.
- Open the **CloudWatch Alarm** to see which metric triggered and the breach value.
- Check **CloudWatch Dashboard** for the affected resource.
- Open **CloudWatch Logs Insights** and search for ERROR patterns around the alarm time.
- Check for any **deployments or scheduled jobs** that ran recently.
- Assess **user impact** – is the service degraded or down?
- If down: escalate and begin **rollback or failover** procedure.
- After resolution: write an **incident summary / postmortem** for the team.

Q9. What Steps Do You Follow During an Incident?

Incident Response Workflow

- **1. Detect:** Alert fires (PagerDuty, Slack). Confirm it is real.
- **2. Acknowledge:** Claim the incident. Notify the team.
- **3. Triage:** What is the severity? Who is impacted?
- **4. Investigate:** Check dashboards, logs, recent deployments.
- **5. Mitigate:** Rollback, scale up, redirect traffic, enable feature flags.
- **6. Resolve:** Confirm metrics have returned to normal.
- **7. Communicate:** Update stakeholders throughout.
- **8. Postmortem:** Root cause analysis, action items, preventive measures.

✓ **Key: Mitigate first (restore service), root cause later (understand why).**

Q10. What is a Postmortem?

A postmortem (or incident review) is a structured analysis done after an incident to understand what happened, why, and how to prevent recurrence.

- **What happened** – Timeline of events.
- **Root cause** – Why did it happen?
- **Impact** – How many users were affected? For how long?
- **Detection** – How was it found? How long did MTTD take?
- **Action items** – What changes will prevent this next time?

✓ **Postmortems are blameless — the goal is to improve the system, not blame individuals.**

Quick Reference Cheat Sheet

Concept	One-Line Definition
Monitoring	Watching known metrics for expected problems
Observability	Understanding WHY unknown failures happen
SLI	What you measure (e.g., 99.2% success rate)
SLO	What you target internally (e.g., $\geq 99.5\%$)
SLA	What you promise contractually (e.g., 99% uptime)
MTTD	Avg time to detect an incident
MTTR	Avg time to recover from an incident
Counter	Always-increasing metric (total requests)
Gauge	Current value metric (CPU %, active connections)
Histogram	Distribution of values in buckets (latency ranges)
rate()	Per-second avg rate of a counter over time range
irate()	Instantaneous rate using last 2 data points
kube-state-metrics	Exposes K8s object states (pod phases, replicas)
Node Exporter	Exposes Linux host hardware metrics
Alertmanager	Routes & deduplicates Prometheus alerts
Remote Write	Prometheus sends metrics to external storage
ILM	Elasticsearch index lifecycle: Hot→Warm→Cold→Delete
Alert Fatigue	Too many alerts causing teams to ignore them
Postmortem	Blameless analysis of incidents to prevent recurrence